Mean Squared Error

Błąd średniokwadratowy
w uczeniu maszynowym

Marta Marcinkowska, Jakub Wanat

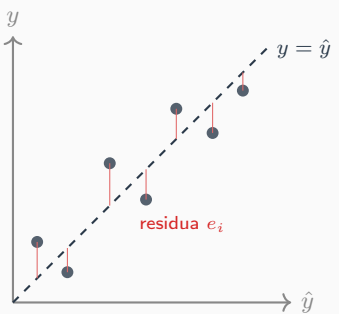
Funkcja straty — po co ją minimalizujemy?

Problem: model przewiduje, rzeczywistość weryfikuje. Potrzebujemy miary rozbieżności.

- Funkcja straty = miara błędu modelu
- Uczenie = **minimalizacja** tej miary
- MSE karze silniej za duże błędy (kwadratowanie)
- Stosowana w problemach **regresji ciągłej**

Przykład: przewidywanie cen mieszkań — MSE mierzy kwadratową różnicę ceny przewidywanej i rzeczywistej.

Źródło: pl.eitca.org



Wyprowadzenie wzoru na MSE

Krok 1 — residuum

$$e_i = y_i - \hat{y}_i$$

Różnica: obserwacja vs. predykcja.

Krok 2 — kwadrat

$$e_i^2 = (y_i - \hat{y}_i)^2$$

Eliminuje znak; penalizuje duże błędy.

Krok 3 — suma kwadratów (SSE)

$$\mathrm{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Krok 4 — uśrednienie

$$\mathrm{MSE} = \frac{\mathrm{SSE}}{n}$$

Finalny wzór

$$\boxed{\mathrm{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

n — liczba obserwacji; y_i — wartość rzeczywista; $\hat{y}_i$ — wartość przewidywana

Zastosowanie: pościg wilka za zającem

Dwa podejścia do predykcji trajektorii — wspólna miara: MSE

1. Regresja wielomianowa

Układ parametryczny:

$$\begin{cases} x(t) = \sum_k a_k t^k \\ y(t) = \sum_k b_k t^k \end{cases}$$

Dopasowanie krzywej do historycznych pozycji.

2. Regr. wielom. zespolona

Izomorfizm:

$$\mathbb{R}^2 \cong \mathbb{C}$$

$$z(t) = x(t) + i\, y(t)$$

Elegancka algebra trajektorii na płaszczyźnie zespolonej.

MSE kwantyfikuje, jak blisko jesteśmy prawdziwej trasy zająca.

Ograniczenia fizyczne trajektorii zająca

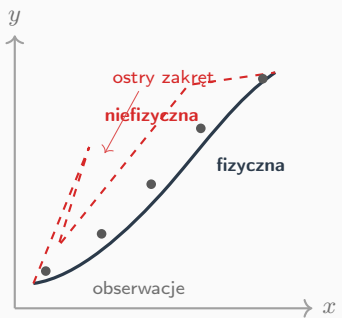
Realny model nie ignoruje biomechaniki — MSE mierzy odchylenia od fizycznie wiarygodnej ścieżki

Dlaczego fizyka ma znaczenie?

- Prędkość i przyspieszenie są **ograniczone biologicznie**:

$$\|\mathbf{v}(t)\| \leq v_{\max}, \quad \|\mathbf{a}(t)\| \leq a_{\max}$$

- Trajektoria musi być **gładka** (ciągła pochodna)
- Nagłe zakręty $\Rightarrow$ odrzucamy je



Model ruchu zająca — klasa `lokalizacja_zajaca`

Inicjalizacja położenia, prędkości i losowego kierunku startu; aktualizacja przez macierz rotacji z szumem gaussowskim

```python
class lokalizacja_zajaca:
    def __init__(self,poczatkowy_x=0.0,poczatkowy_y=0.0,speed=15.0,szum=0.1):
        self.polozenie=np.array([poczatkowy_x,poczatkowy_y])
        self.poczatkowy_kat=np.random.uniform(0,2*pi)
        self.predkosc=speed
        self.szum=szum
        self.wektor_przemieszczenia=np.array([np.sin(self.poczatkowy_kat),np.cos(self.poczatkowy_kat)])*self.predkos

    def kolejne_polozenia(self):
        kat_obrotu=np.random.normal(0,self.szum)
        macierz_przemieszczenia = np.array([
            [np.cos(kat_obrotu), -1 * np.sin(kat_obrotu)],
            [np.sin(kat_obrotu), np.cos(kat_obrotu)]
        ])
        self.wektor_przemieszczenia = macierz_przemieszczenia.dot(self.wektor_przemieszczenia)
        self.polozenie=self.polozenie+self.wektor_przemieszczenia

        return(self.polozenie)
```

Model 1: Regresja wielomianowa osobna

Dwa niezależne wielomiany — jeden dla osi x, drugi dla osi y

```python
#wyznaczam wspolrzedne do dwóch równań
# 1) opisuje położenie względem osi x po t czasie
# 2) opisuje położenie względem osi y po t czasie
wspolrzedna_x=np.polyfit(t,x,stopien)
wspolrzedna_y=np.polyfit(t,y,stopien)

#tworze model odpowiednio dla x i y
model_x=np.poly1d(wspolrzedna_x)
model_y=np.poly1d(wspolrzedna_y)
#obliczam jakie są wartości zgodnie z przyjętym modelem
wartosc_x_wedlug_modelu=model_x(t)
wartosc_y_wedlug_modelu=model_y(t)

#błędy średniokwadratowe
MSE_x=np.mean((x-wartosc_x_wedlug_modelu)**2)
MSE_y=np.mean((y-wartosc_y_wedlug_modelu)**2)

MSE=MSE_x+MSE_y
```

Idea:

- Dwa niezależne dopasowania:

$$x(t) = \sum_k a_k t^k$$

$$y(t) = \sum_k b_k t^k$$

- MSE sumuje błędy obu osi:

$$\mathrm{MSE} = \mathrm{MSE}_x + \mathrm{MSE}_y$$

- Prosty, interpretowalny model

Model 2: Regresja wielomianowa zespolona

Jeden wielomian zespolony — trajektoria jako krzywa w $\mathbb{C}$

```
wspolczynniki = np.polyfit(t, z, stopien)
model_zespolony = np.poly1d(wspolczynniki)
# Wartości przewidziane przez model
wartosci_modelu = model_zespolony(t)
# Błąd średniokwadratowy
MSE = np.mean(np.abs(z - wartosci_modelu)**2)
```

Idea:

- Izomorfizm $\mathbb{R}^2 \cong \mathbb{C}$:

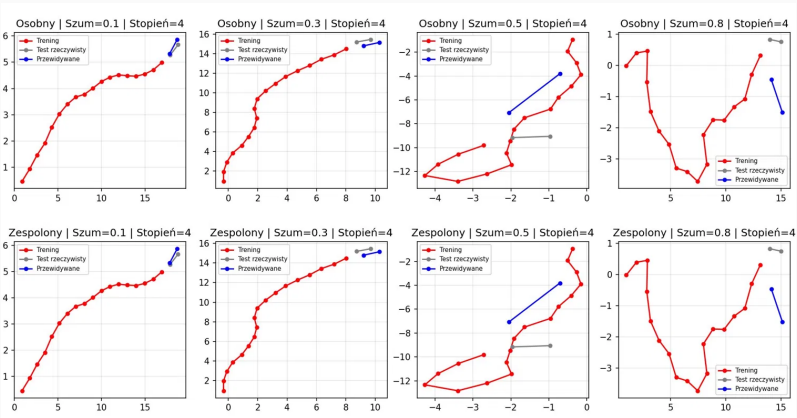
$$z(t) = x(t) + i\, y(t)$$

- Jeden wielomian opisuje **obie osie**
- MSE przez moduł:

$$\mathrm{MSE} = \frac{1}{n} \sum |z_i - \hat{z}_i|^2$$

Porównanie modeli — wpływ szumu i stopnia wielomianu

Górny rząd: model osobny Dolny rząd: model zespolony Stopień $= 4$, szum $\in \{0.1, 0.3, 0.5, 0.8\}$



Generowanie trajektorii — macierz obrotu i szum

Macierz obrotu 2D

Dla kąta θ obrotu w płaszczyźnie:

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

Działanie: obraca wektor prędkości o kąt θ w każdym kroku.

Dlaczego delty, nie współrzędne?

- $\Delta p_t = p_t - p_{t-1}$ — przyrost pozycji
- Delty są **niezależne od początku układu**
- Ułatwiają uczenie: sieć przewiduje **ruch**, nie położenie
- Mniejsza skala $\Rightarrow$ szybsza zbieżność

Generowanie kolejnych kroków

$$\mathbf{v}_{t+1} = R(\theta)\,\mathbf{v}_t + \boldsymbol{\varepsilon}_t$$

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{v}_{t+1}$$

gdzie $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$ — szum normalny.

Wektor przyspieszenia

- $\mathbf{a}_t = \mathbf{v}_{t+1} - \mathbf{v}_t$ — zmiana prędkości
- Wpływa na krzywiznę toru
- $\theta > 0$: skręt w lewo; $\theta < 0$: skręt w prawo

Szum normalny

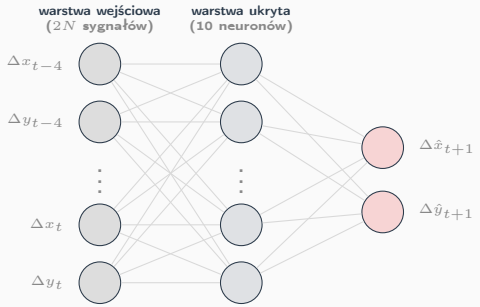
Modeluje nieprzewidywalne zachowanie zająca.
σ kontroluje stopień losowości trajektorii.

Sieć neuronowa jako uniwersalny aproksymator

Aproksymacja nieliniowa

Sieć neuronowa w ujęciu formalnym działa jako funkcja parametryczna $f_\theta : \mathbb{R}^{2N} \to \mathbb{R}^2$, mapująca historię stanów na wektor przyszłego przesunięcia.

- **Przetwarzanie sekwencyjne:** Analiza dynamiki ruchu poprzez kroczące okno czasowe (N ostatnich kroków).
- **Ekstrakcja wzorców:** Sieć izoluje deterministyczny trend (bezwładność) od stochastycznego szumu.
- **Cel optymalizacji:** Uczenie się nieliniowych zależności reprezentujących strategię uników ofiary.



Mapowanie przestrzeni: $\mathbf{X}_t \to \Delta\hat{\mathbf{p}}_{t+1}$

Dobór narzędzia: Klasa MLPRegressor

Uzasadnienie metodologiczne

Wykorzystanie gotowej implementacji (`scikit-learn`) pozwala skupić się na **analizie funkcji straty** w czasie rzeczywistym. Stanowi stabilne środowisko do badania zbieżności gradientowej.

Mechanika wewnętrzna sieci

- **Automatyczna optymalizacja:** Model samodzielnie ewaluuje MSE jako domyślną funkcję straty.
- Wykorzystuje **gradienty** wyznaczone propagacją wsteczną do poszukiwania minimum i aktualizacji wag.
- Nieliniowe funkcje aktywacji (domyślnie ReLU).

Specyfika optymalizacji online

W dynamicznym środowisku klasyczne algorytmy wykazują tendencję do akumulacji błędu. Rozwiązaniem jest ciągła adaptacja ze stabilizacją wag.

Konfiguracja w projekcie

`hidden_layer_sizes=(10,)`
`warm_start=True`
`max_iter=1` (uczenie przyrostowe)

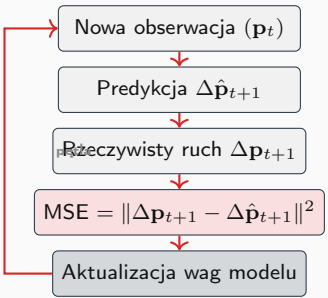
Paradygmat uczenia: Online vs Batch Learning

Uczenie klasyczne (Batch)

- Jednorazowy dostęp do całego zbioru
- **Statyczna minimalizacja ryzyka**
- Problem: Brak dynamiki — model otrzymuje całą ścieżkę naraz zamiast płynnie adaptować się do sytuacji.

Uczenie przyrostowe (Online)

- Wagi modyfikowane **po każdej obserwacji**
- Brak zjawiska wycieku danych.
- Emuluje **realizm pościgu**: drapieżnik uczy się „w locie", reagując na manewry ofiary.



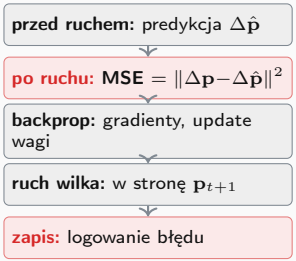
Cykliczna adaptacja z wykorzystaniem `partial_fit()`

Przebieg iteracji i ewaluacja zbieżności

Sekwencja operacyjna kroku t

1. **Wstępna predykcja:** Model prognozuje wektor $\Delta\hat{\mathbf{p}}_{t+1}$ na bazie stanu z okna czasowego.
2. **Generowanie stanu:** Rzeczywiste przesunięcie ofiary $\Delta\mathbf{p}_{t+1}$ w środowisku.
3. **Ewaluacja błędu:** Obliczenie chwilowej wartości MSE między predykcją a stanem faktycznym.
4. **Adaptacja wag:** Informacja zwrotna służy do natychmiastowej korekty macierzy wag.

Fazy przetwarzania w czasie



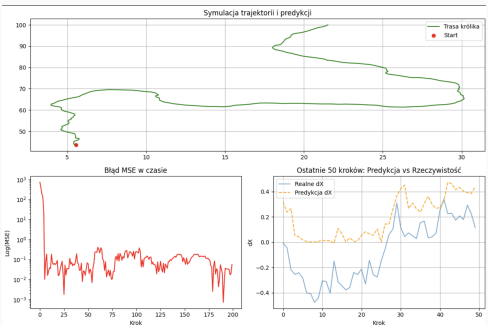
Konwergencja: Każdy gwałtowny zwrot ofiary generuje pik błędu, po którym następuje szybki spadek (adaptacja modelu).

Analiza wyników — panel symulacyjny

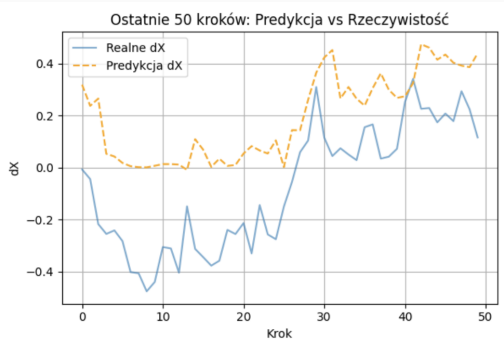
Zintegrowane środowisko

- **Przestrzeń 2D:** Bezpośrednia wizualizacja trasy pościgu.
- **Ewaluacja konwergencji:** Wykres logarytmiczny błędu MSE w kolejnych iteracjach.
- **Analiza składowych:** Rzeczywiste vs. przewidywane przyrosty pozycji (delta x, delta y).

Rozbicie dashboardu na kluczowe komponenty pozwala na rzetelną ocenę działania sieci neuronowej w czasie rzeczywistym.



Dynamika śledzenia trajektorii



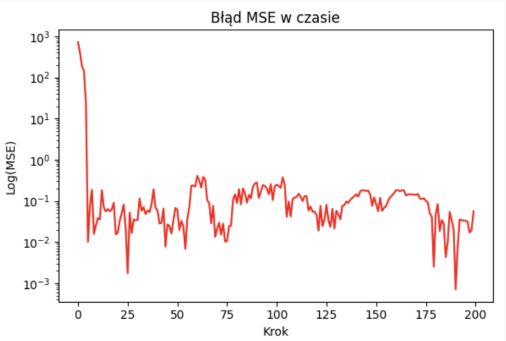
Zdolność adaptacyjna modelu

- Sieć skutecznie naśladuje nieliniową trasę obiektu, mimo wbudowanego szumu stochastycznego.
- **Opóźnienie fazowe (lag):** Widoczna minimalna „bezwładność" predykcji — jest to zjawisko nieuniknione, gdy model reaguje wyłącznie na zaistniałą historię.
- Z zachowania widać, że „drapieżnik" uczy się wzorców ruchu ofiary.

Ewolucja błędu predykcji (Skala Logarytmiczna MSE)

Interpretacja funkcji straty

- **Gwałtowne piki:** Skoki błędu odpowiadają momentom, w których zając wykonuje ostry, nieprzewidywalny skręt.
- **Szybka konwergencja:** Po każdym piku funkcja straty niemal natychmiast opada — to dowód na prawidłowe i szybkie działanie algorytmu propagacji wstecznej.
- Optymalizator skutecznie aktualizuje wagi w stronę nowego minimum lokalnego.



Wnioski końcowe

Sieci neuronowe i MSE w układach dynamicznych

- **Skuteczność Online Learningu:**
  Adaptacja modelu „w locie" jest kluczowa dla środowisk, w których reguły i trajektorie dynamicznie się zmieniają. Klasyczny *batch learning* nie oddałby poprawnie tego aspektu.

- **MSE jako bezwzględny korektor:**
  Błąd średniokwadratowy mocno penalizuje duże odchylenia. Dzięki temu sieć zostaje „zmuszona" do błyskawicznej korekty wag przy każdej próbie zgubienia pościgu przez ofiarę.

- **Wydajność MLP:**
  Mimo relatywnie prostej architektury (tylko 1 warstwa ukryta, 10 neuronów), perceptron wielowarstwowy radzi sobie ze śledzeniem szumu gaussowskiego na zadowalającym poziomie.

Bibliografia (1/2)

- **SAP Polska:** *Czym jest uczenie maszynowe?*
  sap.com/poland/resources/what-is-machine-learning
- **Wikipedia:** *Macierz obrotu*
  pl.wikipedia.org/wiki/Macierz_obrotu
- **SAS Polska:** *Sieci neuronowe — przegląd*
  sas.com/pl_pl/insights/analytics/neural-networks.html
- **pedropAI:** *Machine Learning Types | Batch Learning VS Online Learning*
  youtube.com/watch?v=HTGCgErym1E

Bibliografia (2/2)

- **Machine Learning Made Simple:** *Understanding Online Training*
  `machine-learning-made-simple.medium.com/...`
  `.../understanding-online-training-for-ml-engineers-e7136b57a91e`
- **Scikit-learn Documentation:** *MLPRegressor*
  `scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html`
- **Inspiracje wizualne (grafiki TikZ):**
  `texample.net`